

DC MOTOR POZÍCIÓSZABÁLYOZÁSA PID ÉS FUZZY ALAPÚ SZIMULÁCIÓS MODELLEKKEL

POZICIONA REGULACIJA DC MOTORA KORIŠĆENJEM PID I FUZZY SIMULACIONIH MODELA

POSITION CONTROL OF A DC MOTOR USING PID AND FUZZY-BASED SIMULATION MODELS

Hallgató

NAGY RICHÁRD
25223020

Mentor

DR. PIROSKA STANIĆ MOLCER

Szabadka, 2025

Absztrakt

Ebben a munkában egy egyenáramú (DC) motor pozíciószabályozásának vizsgálata került elvégzésre két különböző irányítási megközelítés alkalmazásával: a hagyományos PID szabályzóval és a fuzzy logikán alapuló szabályozással. A szabályzók működésének értékelése egy Python nyelven megvalósított szimulációs környezetben történt, amelyben valóság-hű motormodell, enkóderszenzor és grafikus megjelenítés került felhasználásra. A modell segítségével bemutatásra került a két irányítási módszer dinamikus viselkedése, stabilitása és beállási tulajdonságai. A kapott eredmények alapján megállapítást nyert, hogy mindkét szabályzó képes a kívánt pozíció pontos követésére, azonban a fuzzy rendszer rugalmasabb működést mutatott nemlineáris körülmények között, míg a PID szabályzó egyszerűbben hangolható és kiszámíthatóbb viselkedést biztosított. A kidolgozott szimulációs környezet alkalmasnak bizonyult oktatási és prototípus-fejlesztési célokra is, mivel lehetővé teszi különböző paraméterek és szabályozási módszerek hatékony összehasonlítását.

Kulcsszavak:

DC motor, fuzzy logika, pozíciószabályozás, PID szabályzó, szimuláció

Tartalom

Absztrakt.....	2
Tartalom	3
Jelmagyarázat.....	4
Köszönetnyilvánítás.....	5
1. Bevezető.....	6
2. DC Motor És Enkóder Működése.....	7
2.1. DC Motor Működése	7
2.2. H-Híd Kapcsolás	7
2.3. DC motor matematikai modellje	8
2.4. Enkóder Működése.....	11
3. Fuzzy Irányítás	12
3.1. Ember és Fuzzy Logika	12
3.2. Fuzzy irányítás alapjai.....	13
4. PID Irányítás.....	15
4.1. PID felépítése.....	15
4.2. Valós PID irányítás	16
5. Python környezet telepítése.....	17
6. Szimulációs szoftver telepítése és futtatása.....	18
7. Szoftver felépítése.....	20
8. Komponensek működése	22
8.1. DC Motor Model.....	22
8.2. Enkóder Model	22
8.3. Fuzzy Szabályzó.....	22
8.4. PID Szabályzó.....	23
8.5. Vizualizációs.....	23
8.6. Motor Paraméterek.....	23
9. Fuzzy Szimuláció működése.....	24
9.1. Univerzumok definiálása	24
9.2. Tagsági Függvények.....	25
9.3. Szabálybázis.....	26

9.4.	Integráló tag és végső kimenet.....	27
9.5.	Szimulációs ciklus.....	28
9.6.	Eredmények vizualizációja.....	28
10.	PID Szimuláció működése.....	30
10.1.	Szabályozó paraméterei.....	30
10.2.	Szimulációs ciklus.....	30
10.3.	Különbségek a fuzzy szimulációhoz képest.....	31
10.4.	Eredmények vizualizációja.....	31
11.	Összegzés és jövőbeli irányok.....	34
11.1.	Rendszer értékelése.....	34
11.2.	Jövőbeli fejlesztési irányok.....	34
11.3.	Szimuláció korlátai.....	36
11.5.	Projekt Összegzése.....	36
12.	Idegen nyelvű tartalmi összefoglaló.....	37
13.	Felhasznált irodalom.....	38

Jelmagyarázat

Jel/Rövidítés	Értelmezés
DC	Egyenáramú motor (angolul: Direct Current motor)
PID	Proporcionális-Integráló-Deriváló szabályzó (angolul: Proportional-Integral-Derivative controller)
Fuzzy	Fuzzy logikán alapuló szabályzó (Fuzzy Logic Controller)
PPR	Impulzus per fordulat, az enkóder felbontása (Pulses Per Revolution)
PWM	Impulzusszélesség-moduláció, motorvezérlésre használt jel (Pulse Width Modulation)
H-híd	Motorirányváltó teljesítménykapcsolás (angolul: H-Bridge)

Köszönetnyilvánítás

Szeretnék köszönetet mondani mindazoknak, akik támogatásukkal hozzájárultak ennek a projektnek a megvalósításához. Külön köszönöm Dr. Piroska Stanić Molcer-nek a szakmai útmutatást és a hasznos tanácsokat, amelyek nagyban segítettek a munkám fejlődését. Hálával tartozom a családomnak is, akik folyamatosan biztattak és biztosították a szükséges háttérrel a projekt elkészítéséhez. Támogatásuk nélkül ez a munka nem jöhetett volna létre.

1. Bevezető

A modern ipari automatizálás és robotika fejlődésével az irányítástechnikai rendszerek egyre fontosabb szerepet töltenek be mindennapi életünkben. A pontos pozicionálás, sebesség- és nyomatékszabályozás elengedhetetlen követelménye számos alkalmazásnak, legyen szó akár egy ipari robotkar mozgatásáról, egy CNC megmunkológép szerszámvezetéséről, vagy egy elektromos jármű hajtásláncának vezérléséről. Ezen feladatok megoldásában kulcsszerepet játszanak az egyenáramú (DC) motorok, amelyek egyszerű felépítésük, könnyen szabályozható jellemzőik és széles körű elérhetőségük miatt az ipar egyik leggyakrabban alkalmazott hajtástípusává váltak.

Az irányítástechnika területén számos szabályozási módszer áll rendelkezésre a kívánt rendszerviselkedés elérésére. A klasszikus megközelítések közül kiemelkedik a PID (Proporcionális-Integráló-Deriváló) szabályozó, amely matematikailag jól megalapozott, könnyen hangolható és robusztus működést biztosít. A PID szabályozók évtizedek óta bizonyítják hatékonyságukat az ipari környezetben, és ma is a legszélesebb körben alkalmazott visszacsatolt irányítási megoldásnak számítanak.

Az 1960-as évektől kezdődően azonban új irányítási paradigma jelent meg Lotfi Zadeh professzor munkásságának köszönhetően: a fuzzy logika. Ez a megközelítés lehetővé teszi, hogy a hagyományos kétértékű (igaz-hamis) logika helyett az emberi gondolkodáshoz hasonló, fokozatos átmeneteket kezelő szabályozást valósítsunk meg. A fuzzy irányítás különösen hatékony olyan rendszereknél, amelyek pontos matematikai modellje nehezen meghatározható, vagy ahol az emberi szakértői tudás beépítése előnyös lehet a szabályozásba.

Jelen projekt célja egy DC motor pozíciószabályozó rendszer szimulációs környezetének bemutatása, amely lehetővé teszi mind a fuzzy logikán alapuló, mind a hagyományos PID szabályozási stratégiák vizsgálatát és összehasonlítását. A Python programozási nyelven megvalósított szoftver valósághű motormodellt, enkóder szenzort és vizualizációs eszközöket tartalmaz, amelyek segítségével a felhasználó részletesen tanulmányozhatja a különböző irányítási megközelítések működését és teljesítményét.

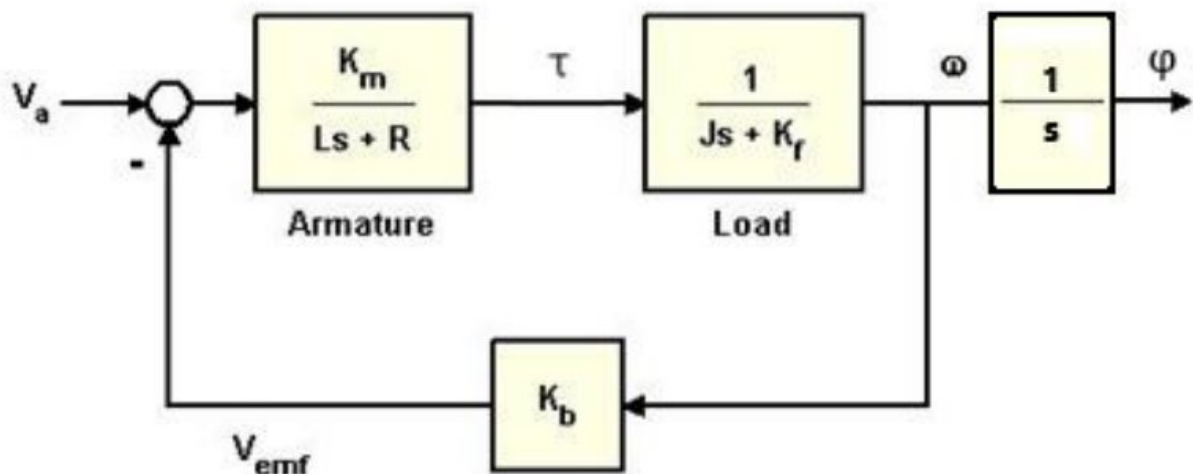
A szimulációs környezet oktatási és prototípus-fejlesztési célokat egyaránt szolgál. Segítségével a felhasználó valós idejű visszacsatolást kap a szabályozók működéséről, vizuálisan követheti a tagsági függvények hatását, és kísérletezhet a különböző paraméterbeállításokkal anélkül, hogy fizikai eszközökre lenne szüksége. Ez a megközelítés ideális alapot biztosít a szabályozástechnika mélyebb megértéséhez és további fejlesztések megvalósításához.

2. DC Motor És Enkóder Működése

2.1 DC Motor Működése

Az egyenáramú (DC) motor az egyik legelterjedtebb villamos hajtás, amelyet egyszerű felépítése és könnyű szabályozhatósága miatt széles körben alkalmaznak ipari és fogyasztói berendezésekben. A DC motor működése az elektromágneses indukción alapul: a rotor és a stator mágneses terének kölcsönhatása hozza létre a forgómozgást. A forgórészen elhelyezett armatúratekercsekben folyó áram mágneses mezőt hoz létre, miközben a kommutátor folyamatosan megfordítja az áram irányát, így biztosítva a rotor folyamatos forgását.

A DC motorok fontos előnye, hogy sebességük és forgásirányuk egyszerűen szabályozható. A fordulatszám a motorra jutó átlagos feszültség módosításával állítható – gyakorlati megvalósításban ez pulzusszélesség-modulációval (PWM) történik. A forgásirány a tápfeszültség polaritásának megfordításával változtatható. A precíz vezérléshez motorvezérlő áramkörre van szükség, amely a motoráramot és a kapocsfeszültséget a kívánt üzemi paraméterekhez igazítja.



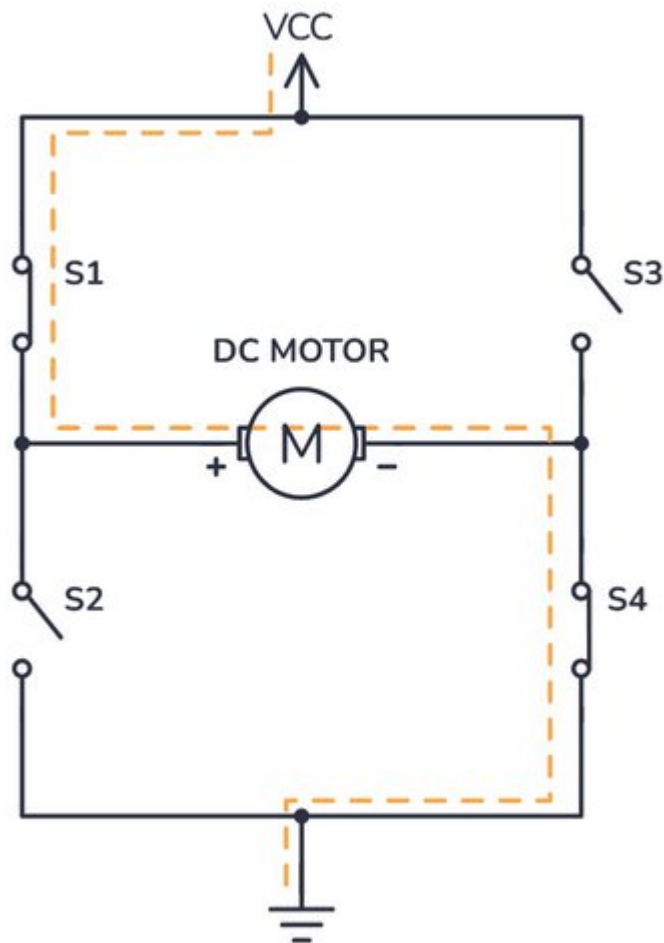
1. Ábra: DC Motor Általános Modellje

2.2 H-Híd Kapcsolás

A DC motorok irányításának egyik legelterjedtebb módja a H-híd (H-Bridge) áramkör. A kapcsolás négy félvezető elemből – általában tranzisztorokból vagy MOSFET-ekből – épül fel, amelyek H-alakú elrendezésben helyezkednek el. A H-híd működése az áramirány megfordításán alapul: a megfelelő két átlós kapcsoló bekapcsolásával a motoron átfolyó áram iránya megváltozik, így a forgásirány szabályozható.

Ha az egyik diagonális kapcsolópárt aktiváljuk, a motor az óramutató járásával megegyező irányba forog; a másik diagonális pár bekapcsolásával pedig az ellentétes irányba. A két felső vagy két alsó kapcsoló egyidejű bekapcsolása tilos, mert rövidzárat okozna a táp felé.

A motor fordulatszáma pulzusszélesség-modulációval (PWM) szabályozható: a kitöltési tényező módosításával az átlagos feszültség változik, így a motor sebessége finoman állítható.



2. Ábra: H-híd kapcsolás

2.3 DC motor matematikai modellje

A szimulációban a DC motor viselkedését differenciálegyenletekkel írjuk le, amelyek az elektromos és mechanikai alrendszert modellezzik.

$$H(s) = \frac{\Phi(s)}{V_{app}(s)} = \frac{K_m}{s[(Js + K_f)(Ls + R) + K_b K_m]}$$

$v_{app}(t)$ – kapocsfeszültség

$\tau(t)$ – forgató nyomaték.

$i(t)$ – a forgórész árama

$\omega(t)$ – A forgórész szögsebessége.

L – a forgórész induktivitása

K_f – súrlódási együttható.

R – a forgórész ellenállása

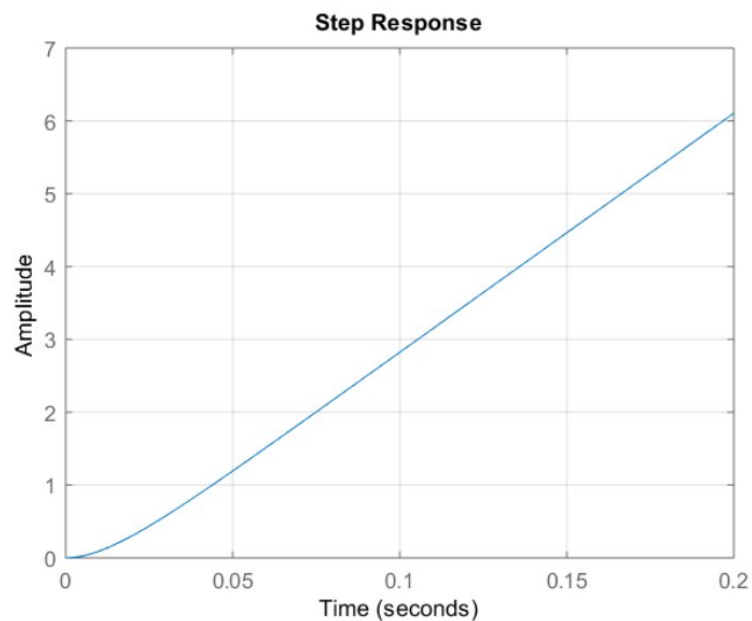
J – tehetetlenségi nyomaték

K_b – elektromos állandó

K_m – motorállandó

Rendszer Matlab Szimulációja

```
s = tf('s');  
J=3.2E-6;  
Kf=3.5E-6;  
Km=0.03;  
Kb=Km;  
R=4; L=3E-6;  
H1 = tf(Km,[L R]);  
H2 = tf(1,[J Kf]);  
P_motor = Km/(s*((J*s+Kf)*(L*s+R)+Km*Kb));  
step(P_motor,0.2)  
grid;
```



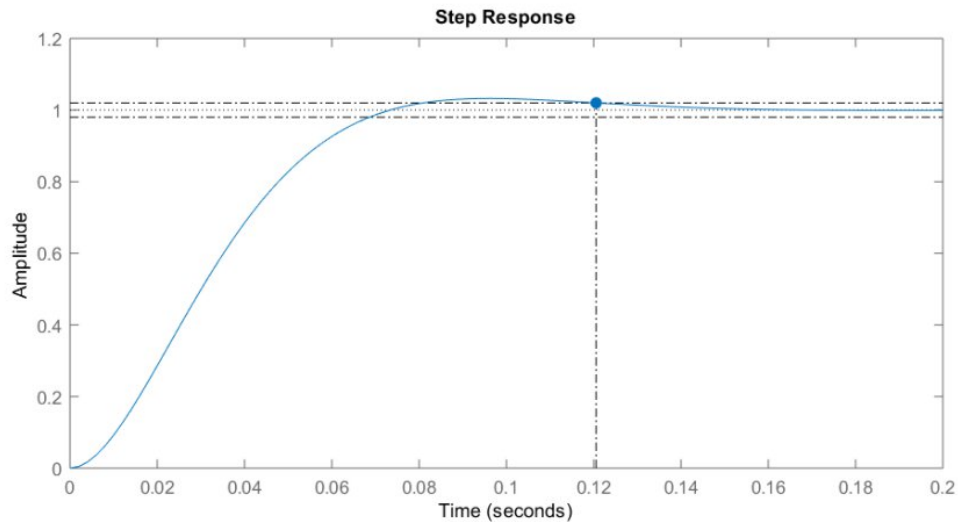
3 Ábra: DC motor rendszer szimulációja

Mivel a rendszer integráló jellegű, nem stabil, egységugrásra adott válasza egy folyamatosan növekvő értékű jel, mely összhangban van azzal, hogyha egy egyenáramú motorra egy fix feszültséget kapcsolunk, a tengely elfordulási szöge folyamatosan nőni fog.

Mereven Vissza csatolt rendszer

```
sys_cl = feedback(P_motor,1)
```

```
step(sys_cl,0.2)
```



4. Ábra: DC motor vissza csatolt rendszer szimulációja

damp(sys_cl)			
Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-3.57e+01 + 3.27e+01i	7.37e-01	4.84e+01	2.80e-02
-3.57e+01 - 3.27e+01i	7.37e-01	4.84e+01	2.80e-02
-1.33e+06	1.00e+00	1.33e+06	7.50e-07

5. Ábra: Rendszer tulajdonságai

A zárt rendszer egy domináns komplex konjugált gyökpárral rendelkezik, melyek időállandója 28 ms, illetve egy nagyon gyors, nagyon kicsi (0.75 us) időállandójú valós gyökkel.

2.4 Enkóder Működése

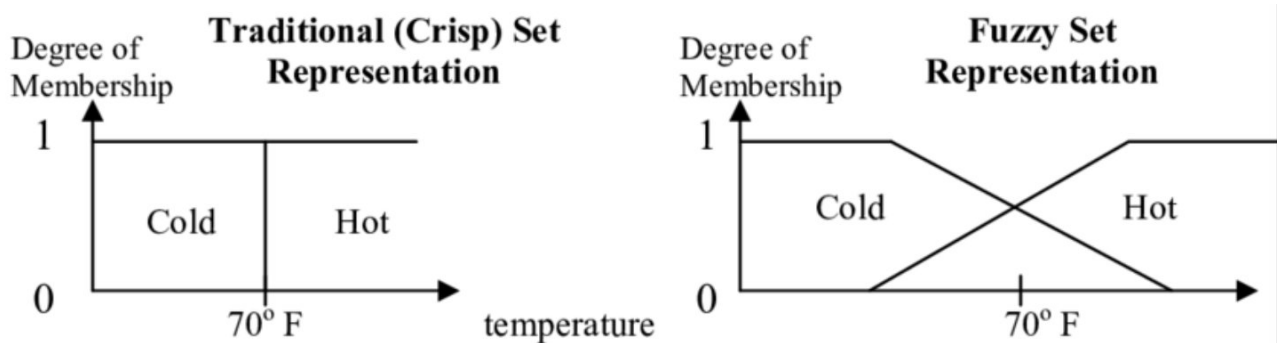
A rotációs enkóder a tengely szögelfordulását digitális impulzusjelekké alakítja. Működése optikai vagy mágneses érzékelésen alapul: a forgó réselt (vagy mágnesesen jelölt) tárcsa elhaladásakor minden jelölés egy impulzust hoz létre. Az enkóder felbontását a PPR (pulses per revolution) érték adja meg. Egy 1000 PPR-es inkrementális enkóder 0,36°/impulzus szögfelbontást biztosít, amely négyszerezett számlálással (A és B csatorna fel- és lefutó élei) akár 0,09°-ra is finomítható.

A folytonos szög diszkrét impulzusokká történő alakítása kvantálási hibát eredményez, amely tipikusan $\pm 0,5$ impulzusnyi. Az A és B fázisjelek 90°-os fázistolása lehetővé teszi a forgásirány meghatározását. Az enkóder jelei pozíció-visszacsatolásként szolgálnak, így zárt hurkú szabályozás valósítható meg.

3. Fuzzy Irányítás

3.1 Ember és Fuzzy Logika

Az emberi gondolkodás és döntéshozatal alapvetően eltér a hagyományos digitális logikától. Míg a klasszikus Boole-algebra kizárólag két állapotot ismer – igaz vagy hamis, 0 vagy 1 –, addig az emberek természetes módon kezelnek olyan fogalmakat, amelyek nem sorolhatók egyértelműen egyik kategóriába sem. Jó példa erre a hőmérséklet leírása: „kicsit hideg van”, „elég meleg”, „nagyon forró”. Ezek a kifejezések nem éles határvonalakat jelölnek, hanem folyamatos átmeneteket, amelyek a valóság homályos, fokozatos jellegét tükrözik.



6 Ábra: Crisp és Fuzzy Érték Ábrázolása

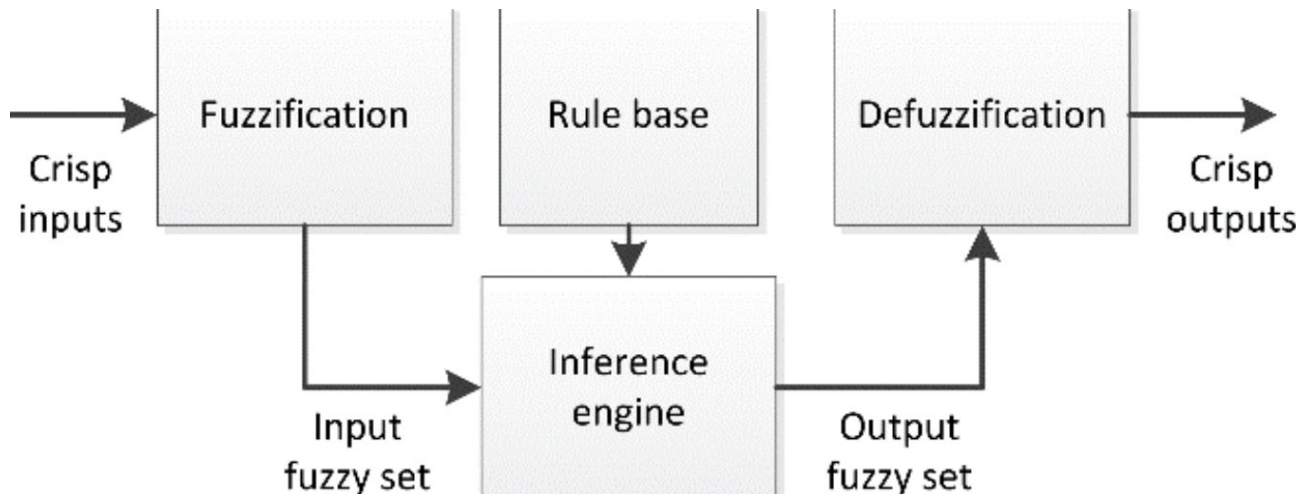
Lotfi A. Zadeh, a Berkeley Egyetem professzora 1965-ben publikálta úttörő munkáját a fuzzy halmazokról. Felismerte, hogy a valós világ jelenségeinek leírásához olyan matematikai eszközre van szükség, amely képes kezelni a fokozatosságot és a fogalmak homályos határait. A fuzzy logika alapgondolata, hogy egy elem nem egyszerűen tartozik vagy nem tartozik egy halmazhoz; a tagság mértéke 0 és 1 közötti értékkel fejezhető ki.

Vegyünk egy hétköznapi példát: egy autó sebességének megítélését. A hagyományos logika szerint meg kellene határoznunk egy éles határt, például azt, hogy „gyors az, ami 100 km/h felett van”. De mi a helyzet 99 km/h-val? Az emberi gondolkodás szerint ez lehet „majdnem gyors”, vagy „elég gyors”. A fuzzy logika ezt tagsági függvényekkel modellezi: 99 km/h-hoz például 0.9-es, míg 80 km/h-hoz csupán 0.3-as tagsági fok rendelhető a „gyors” halmazban – a konkrét értékek a rendszer tervezőjétől függenek.

Ez a megközelítés lehetővé teszi, hogy a szakértői tudást – amely gyakran nyelvi szabályok formájában jelenik meg – közvetlenül beépítsük az irányítási rendszerekbe. Egy tapasztalt gépkezelő nem egyenletekben gondolkodik, hanem olyan szabályokban, mint: „ha a hőmérséklet magas és gyorsan emelkedik, akkor nagymértékben csökkentsd a fűtést”. A fuzzy rendszerek éppen ilyen típusú, nyelvi formában megfogalmazott tudás kezelésére képesek.

3.2 Fuzzy irányítás alapjai

A fuzzy szabályozó három fő komponensből épül fel: **fuzzifikációból, következtetési mechanizmusból (inferencia) és defuzzifikációból**. A rendszer feladata, hogy a pontos (numerikus) bemeneti értékeket nyelvi változókká alakítsa, ezeket a fuzzy szabályok segítségével feldolgozza, majd az eredményt ismét egy pontos kimeneti értékévé konvertálja. Ez a struktúra teszi lehetővé, hogy a szabályozó az emberi szakértők által használt nyelvi tudást matematikai formában kezelje és alkalmazza.



7 Ábra: Fuzzy Irányító rendszer felépítése

- **Fuzzifikáció**

A fuzzifikáció során a crisp (éles, numerikus) bemeneti értékeket tagsági fokokká alakítjuk. Minden bemeneti változóhoz – jelen esetben a pozícióhibához és a hiba változásához – tagsági függvényeket definiálunk. Ezek a függvények határozzák meg, hogy egy adott bemeneti érték milyen mértékben tartozik az egyes nyelvi kategóriákhoz (például „negatív”, „nulla”, „pozitív”).

A tagsági függvények alakja lehet háromszög (trimf), trapéz (trapmf), Gauss-görbe vagy más matematikai forma. A háromszög- és trapézalakú tagsági függvények számítási szempontból egyszerűbbek és gyorsabbak, ezért valós idejű szabályozási feladatokban gyakran ezeket alkalmazzák.

- **Szabálybázis és inferencia**

A szabályok száma a bemeneti változók és azok nyelvi értékeinek számától függ. Két bemenet esetén – amelyek mindhárom nyelvi értéket vehetik fel (negatív, nulla, pozitív) – összesen $3 \times 3 = 9$ szabály szükséges ahhoz, hogy a teljes bemeneti tartományt lefedjük.

Az inferencia során ezek a szabályok a bemeneti tagsági fokok függvényében aktiválódnak. Minden szabály „tüzelési szintje” a szabály előzményében szereplő tagsági fokok kombinációjából adódik; az ÉS kapcsolatot tipikusan a minimum operátor segítségével valósítjuk meg. Így a szabály akkor járul hozzá a kimenethez, amilyen mértékben a bemeneti feltételek teljesülnek.

Rule Base (9 rules):

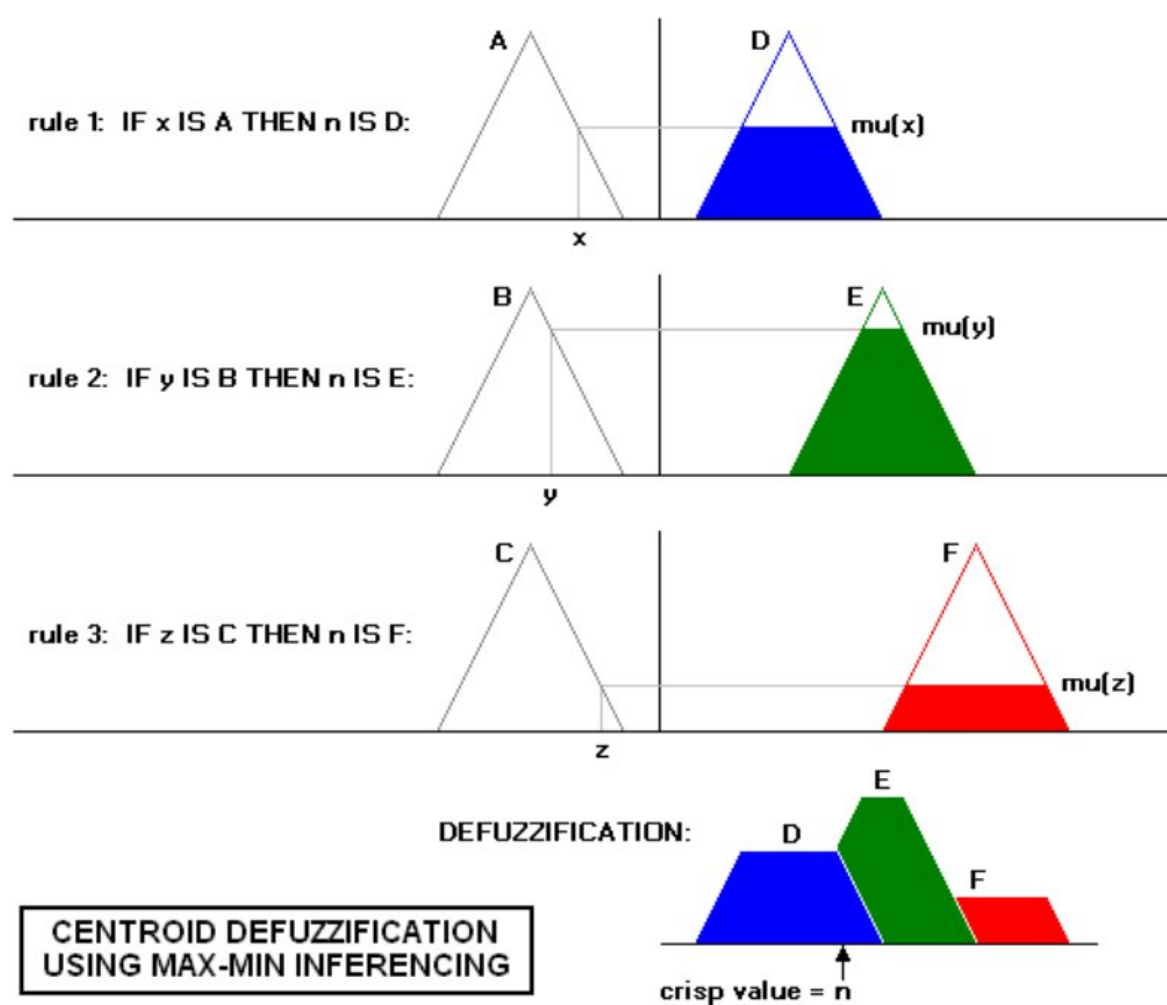
Error \ Delta	Negative	Zero	Positive
Negative	N	N	Z
Zero	Z	Z	Z
Positive	Z	P	P

8 Ábra: Rendszer Szabály Bázisa

- Defuzzifikáció**

Az inferencia eredménye egy fuzzy kimeneti halmaz, amelyet a defuzzifikáció alakít vissza pontos (crisp) kimeneti értéké. A leggyakrabban alkalmazott eljárás a súlypontmódszer (centroid), amely a kimeneti fuzzy halmaz geometriai súlypontját számítja ki, így egy átlagolt, kiegyensúlyozott kimenetet ad.

Alternatív defuzzifikációs módszerek is léteznek, például a maximum középpontja (MOM – Mean of Maximum) vagy a legnagyobb maximum (LOM – Largest of Maximum), amelyek a kimeneti halmaz legnagyobb tagsági értékéhez tartozó pont(ok) alapján választják ki a crisp kimenetet.

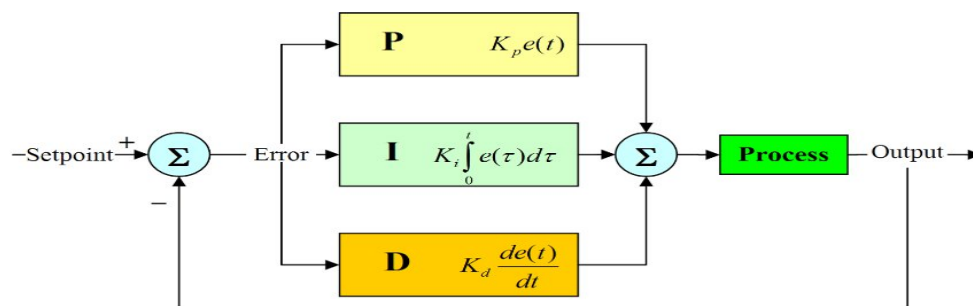


9 Ábra: Defuzzifikáció

4. PID Irányítás

A PID (Proporcionális–Integráló–Deriváló) szabályozó az iparban az egyik legelterjedtebb visszacsatolt irányítási algoritmus. Egyszerű matematikai felépítése ellenére rendkívül hatékony eszköz a dinamikus rendszerek viselkedésének befolyásolására és stabilizálására. A DC motorok szabályozásában a PID algoritmust gyakran alkalmazzák a pozíció, a sebesség vagy a nyomaték pontos beállítására, mivel képes kompenzálni a rendszer hibáit, külső zavarásait és nemlinearitásait.

4.1 PID felépítése



10 Ábra. – PID Szabályzó általános modelje

Proporcionális (P) tag:

A proporcionális tag a pillanatnyi hibával arányos beavatkozást hoz létre: minél nagyobb a hiba, annál nagyobb a kimeneti jel. A P tag gyors reakciót biztosít és javítja a rendszer dinamikáját, azonban önmagában nem képes a maradó hiba teljes megszüntetésére. Túl nagy erősítés esetén a szabályozott rendszer instabillá válhat, és oszcilláció léphet fel.

Integráló (I) tag:

Az integráló tag a hiba időbeli összegét veszi figyelembe. Fő szerepe a tartós, kis hibák (maradó hiba) eltüntetésére, amelyeket a P tag nem képes önmagában kompenzálni. Hátránya, hogy túl nagy integráló erősség a rendszer lassúvá válását, túllendülést és akár instabilitást is okozhat.

Deriváló (D) tag:

A deriváló tag a hiba változási sebességére reagál, vagyis a hiba deriváltját használja a beavatkozás meghatározásához. A D tag kvázi „előrejelzi” a rendszer várható viselkedését, csökkenti a túllendülést és növeli a stabilitást. Ugyanakkor érzékeny a mérési zajra, ami hamis vagy ugráló derivált értékeket eredményezhet, ezért gyakran szűréssel együtt alkalmazzák.

Teljes PID egyenlet

A három tag összegzésével kapjuk a beavatkozójelet:

$$u_{PID}(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

PID Átviteli függvénye:

$$C_{PID}(s) = K_p + K_i \frac{1}{s} + K_d s = A_p \left(1 + \frac{1}{T_i s} + T_d s \right) = \frac{A_p}{T_i} \frac{T_i T_d s^2 + T_i s + 1}{s}$$

4.2 Valós PID irányítás

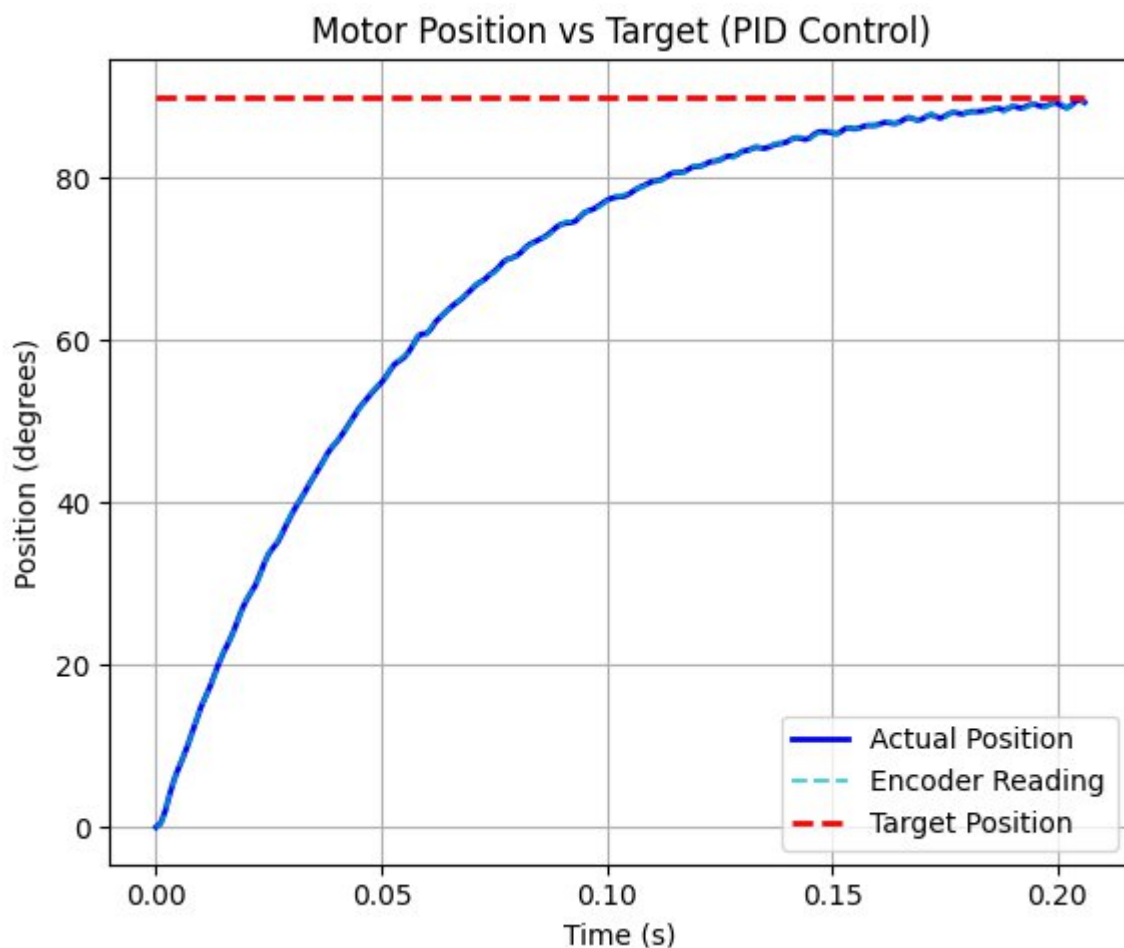
Az ideális PID szabályozó nem realizálható, a valós vagy közelítő PID szabályozóba egy plusz pólust kell beiktatnunk T időállandóval.

Valós PID megvalósítás

$$\hat{C}_{PID}(s) = A_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{Ts + 1} \right) = \frac{A_p}{T_i} \frac{T_i (T_d + T) s^2 + (T_i + T) s + 1}{s(Ts + 1)}$$

$$T = \frac{T_d}{N}$$

ahol N a pólusát helyezési arány.



11 Ábra. – PID Szabályzó ki menete aperiodikus jel esetén

5. Python környezet telepítése

A szimulációs szoftver futtatásához Python 3.x környezet szükséges. A Python egy nyílt forráskódú, platformfüggetlen programozási nyelv, amely kiválóan alkalmas tudományos számításokra, adatfeldolgozásra és szimulációk készítésére.

Python telepítésének ellenőrzése

Linux rendszerek többségén a Python alapértelmezetten telepítve van. A telepített verzió ellenőrzéséhez nyissunk egy terminált, és adjuk ki az alábbi parancsot:

```
python3 --version
```

```
~/Documents/DC-Motor-Position-Control-Fuzzy-Simulation git:(pid-control)±2 (0.043s)
python3 --version
Python 3.12.3
```

12 Ábra: Python Verzió le kérése

Ha a Python nincs telepítve, Ubuntu/Debian alapú rendszereken az alábbi parancsokkal telepíthető:

```
sudo apt update
```

```
sudo apt upgrade
```

```
sudo apt install python3 python3-pip python3-venv
```

A virtuális környezet létrehozását és a szükséges Python-csomagok telepítését a projekt telepítő scriptje automatikusan elvégzi — ennek használatát a következő fejezetben ismertetjük.

```
krishna@pc:~$ sudo apt install python3-pip
[sudo] password for krishna:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libexpat1-dev libpython3.8-dev libpython3.8-dev python-pip-whl python3-dev
  python3-wheel python3.8-dev zlib1g-dev
The following NEW packages will be installed:
  libexpat1-dev libpython3.8-dev libpython3.8-dev python-pip-whl python3-dev
  python3-pip python3-wheel python3.8-dev zlib1g-dev
0 upgraded, 9 newly installed, 0 to remove and 0 not upgraded.
Need to get 6,789 kB of archives.
After this operation, 25.5 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu focal/main amd64 libexpat1-dev amd64 2.2.9-1build1 [116 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 libpython3.8-dev amd64 3.8.5-1-20.04 [3,941 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu focal/main amd64 libpython3.8-dev amd64 3.8.2-0ubuntu2 [7,236 B]
Get:4 http://in.archive.ubuntu.com/ubuntu focal-updates/universe amd64 python-pip-whl all 20.0.2-Subuntu1.1 [1,799 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 zlib1g-dev amd64 1:1.2.11.dfsg-2ubuntu1.2 [155 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 python3.8-dev amd64 3.8.5-1-20.04 [514 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu focal/main amd64 python3-dev amd64 3.8.2-0ubuntu2 [1,212 B]
Get:8 http://in.archive.ubuntu.com/ubuntu focal/universe amd64 python3-wheel all 0.34.2-1 [23.8 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu focal-updates/universe amd64 python3-pip all 20.0.2-Subuntu1.1 [230 kB]
Fetched 6,789 kB in 19s (366 kB/s)
Selecting previously unselected package libexpat1-dev:amd64.
Reading database ... 211064 files and directories currently installed.)
Preparing to unpack .../0-libexpat1-dev_2.2.9-1build1_amd64.deb ...
Unpacking libexpat1-dev:amd64 (2.2.9-1build1) ...
Selecting previously unselected package libpython3.8-dev:amd64.
Preparing to unpack .../1-libpython3.8-dev_3.8.5-1-20.04_amd64.deb ...
Unpacking libpython3.8-dev:amd64 (3.8.5-1-20.04) ...
Selecting previously unselected package libpython3.8-dev:amd64.
Preparing to unpack .../2-libpython3.8-dev_3.8.2-0ubuntu2_amd64.deb ...
Unpacking libpython3.8-dev:amd64 (3.8.2-0ubuntu2) ...
Selecting previously unselected package python-pip-whl.
Preparing to unpack .../3-python-pip-whl_20.0.2-Subuntu1.1_all.deb ...
Unpacking python-pip-whl (20.0.2-Subuntu1.1) ...
Selecting previously unselected package zlib1g-dev:amd64.
Preparing to unpack .../4-zlib1g-dev_1:1.2.11.dfsg-2ubuntu1.2_amd64.deb ...
Unpacking zlib1g-dev:amd64 (1:1.2.11.dfsg-2ubuntu1.2) ...
Selecting previously unselected package python3.8-dev.
Preparing to unpack .../5-python3.8-dev_3.8.5-1-20.04_amd64.deb ...
Unpacking python3.8-dev (3.8.5-1-20.04) ...
Selecting previously unselected package python3-dev.
Preparing to unpack .../6-python3-dev_3.8.2-0ubuntu2_amd64.deb ...
Unpacking python3-dev (3.8.2-0ubuntu2) ...
Selecting previously unselected package python3-wheel.
```

13 Ábra: Python Környezet telepítése

6. Szimulációs szoftver telepítése és futtatása

A szimulációs szoftver telepítése és futtatása shell scriptek segítségével egyszerűen elvégezhető. A projekt két fő scriptet tartalmaz:

- setup.sh, amely a futtatási környezet előkészítéséért felel,
- run_simulation.sh, amely a szimuláció indítását végzi.

Projekt letöltése

A projekt forráskódja a Git verziókezelő rendszer használatával tölthető le:

```
cd DC-Motor-Position-Control-Fuzzy-Simulation
```

Futtatási jogok beállítása

A scriptek futtatásához először végrehajtási jogot kell adni:

```
chmod +x setup.sh run_simulation.sh
```

Környezet telepítése

Ezután futtassuk a **setup.sh** scriptet, amely automatikusan elvégzi a következő lépéseket:

- Virtuális Python-környezet létrehozása
- A pip csomagkezelő frissítése
- A szükséges függőségek telepítése a requirements.txt alapján

A telepítést az alábbi paranccsal indíthatjuk:

```
./setup.sh
```

```
=====
DC Motor Position Control - Setup
=====
Creating virtual environment...
Activating virtual environment...
Upgrading pip...
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (25.3)
Installing required packages from requirements.txt...
Requirement already satisfied: numpy in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 1)) (2.3.5)
Requirement already satisfied: scipy in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 2)) (1.16.3)
Requirement already satisfied: networkx in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 3)) (3.6)
Requirement already satisfied: scikit-fuzzy in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 4)) (0.5.0)
Requirement already satisfied: matplotlib in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 5)) (3.10.7)
Requirement already satisfied: simple-pid in ./venv/lib/python3.12/site-packages (from -r requirements.txt (line 6)) (2.0.1)
Requirement already satisfied: contourpy==1.0.1 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (1.3.3)
Requirement already satisfied: cycler==0.10 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (0.12.1)
Requirement already satisfied: fonttools==4.22.0 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (4.61.0)
Requirement already satisfied: kiwisolver==1.3.1 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (1.4.9)
Requirement already satisfied: packaging==20.0 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (25.0)
Requirement already satisfied: pillow==8 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (12.0.0)
Requirement already satisfied: pyparsing==3 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (3.2.5)
Requirement already satisfied: python-dateutil==2.7 in ./venv/lib/python3.12/site-packages (from matplotlib->-r requirements.txt (line 5)) (2.9.0.post0)
Requirement already satisfied: six==1.5 in ./venv/lib/python3.12/site-packages (from python-dateutil==2.7->matplotlib->-r requirements.txt (line 5)) (1.17.0)
=====
Setup completed successfully!
=====

To activate the virtual environment manually, run:
source venv/bin/activate

To run the simulation, use:
./run_simulation.sh start_position=<value> end_position=<value>
Example: ./run_simulation.sh start_position=-90 end_position=45
```

14 Ábra. – setup.sh script ki menete futtatás során

Szimuláció futtatása

A szimuláció indításához a run_simulation.sh scriptet használjuk, amely a következő paramétereket várja:

start_position	Kezdeti pozíció fokban (-180 és 180 között)	Igen
end_position	Cél pozíció fokban (-180 és 180 között)	Igen
controller	Szabályozó típusa: fuzzy vagy pid	Nem (alapértelmezett: fuzzy)

Fuzzy szabályozóval:

```
./run_simulation.sh start_position=-90 end_position=45 controller=fuzzy
```

PID szabályozóval:

```
./run_simulation.sh start_position=-90 end_position=45 controller=pid
```

```
./run_simulation.sh start_position=0 end_position=90 controller=pid
Activating virtual environment...
=====
Starting DC Motor Position Control Simulation
Start Position: 0 degrees
End Position: 90 degrees
Controller: pid
=====

=====
DC Motor PID Position Controller
with Realistic Motor Physics Model
=====

Starting closed-loop simulation with encoder feedback:
Controller: PID
Encoder resolution: 1000 PPR (0.360°/count)
Encoder noise: 0.1° std dev
Initial position: 0.0°
Target position: 90.0°
Initial error: 90.0183937938029°
Time step: 1.00 ms

Step 20 (0.020s): Actual=27.93°, Measured=28.14°, Err=63.45°, V=-0.24V, Count=78
Step 40 (0.040s): Actual=47.41°, Measured=47.48°, Err=43.18°, V=0.92V, Count=132
Step 60 (0.060s): Actual=60.78°, Measured=60.68°, Err=29.19°, V=3.88V, Count=169
Step 80 (0.080s): Actual=70.36°, Measured=70.23°, Err=19.77°, V=0.33V, Count=195
Step 100 (0.100s): Actual=77.20°, Measured=77.11°, Err=13.40°, V=-0.08V, Count=214
Step 120 (0.120s): Actual=81.37°, Measured=81.46°, Err=8.64°, V=-0.37V, Count=226
Step 140 (0.140s): Actual=84.39°, Measured=84.25°, Err=5.85°, V=2.41V, Count=234
Step 160 (0.160s): Actual=86.48°, Measured=86.31°, Err=3.52°, V=0.07V, Count=240
Step 180 (0.180s): Actual=88.05°, Measured=88.16°, Err=2.21°, V=0.11V, Count=245
Step 200 (0.200s): Actual=89.12°, Measured=89.35°, Err=0.80°, V=-2.28V, Count=248

Converged at step 206 (0.206s)!
Final actual position: 89.26°
Final measured position: 89.22°
Final error: 0.37°
Final velocity: -416.95°/s
Final current: -0.5160 A
Final encoder count: 248

Displaying simulation results...
Displaying final summary...
```

15 Ábra. – run_simulation.sh ki menete futtatás során

7. Szoftver felépítése

A szimulációs szoftver moduláris architektúrával rendelkezik, ahol minden komponens jól elkülönített feladatot lát el. Ez a felépítés megkönnyíti a kód karbantartását, tesztelését és továbbfejlesztését.

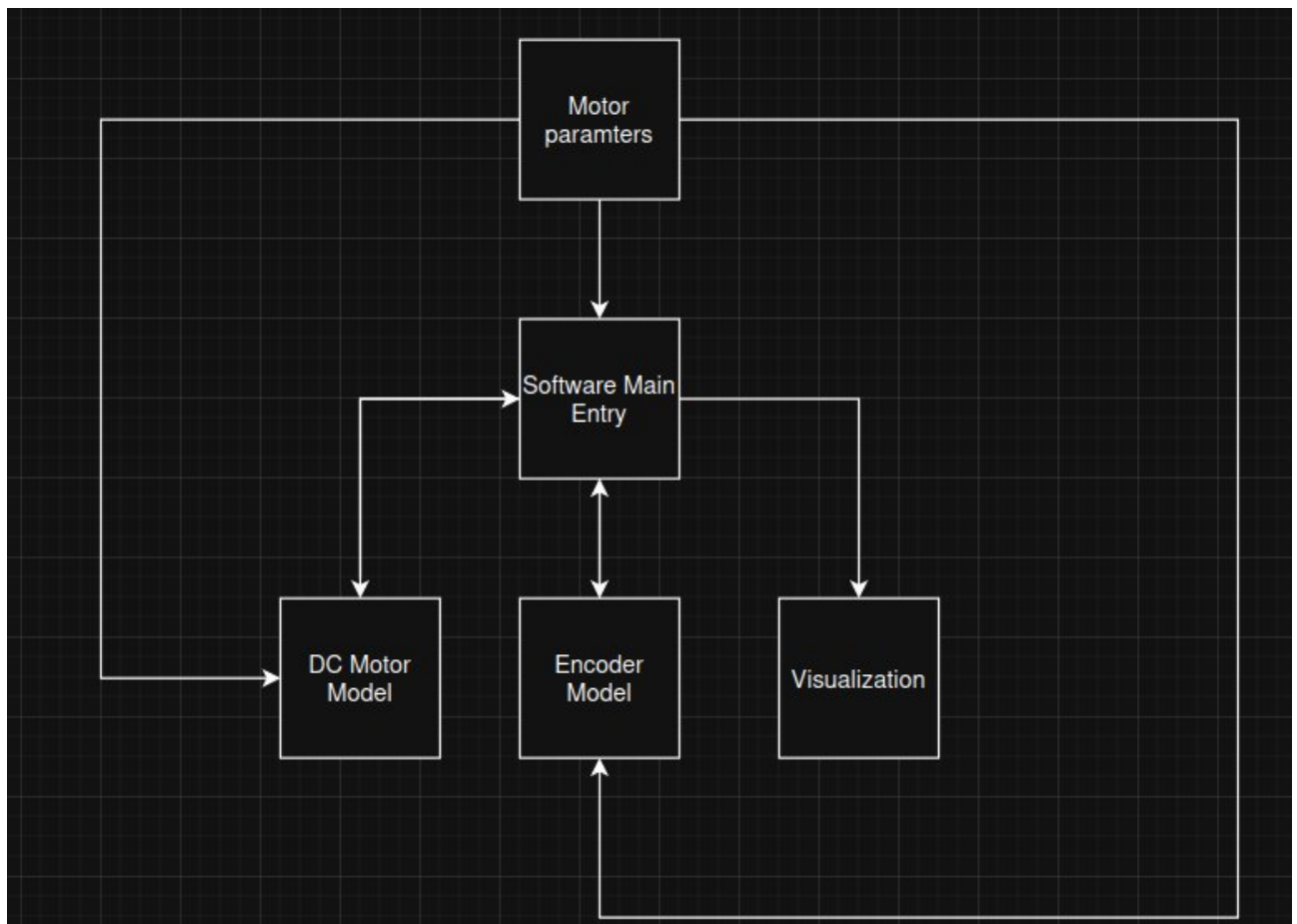
Projekt struktúra

main.py	A program belépési pontja. Feldolgozza a parancssori argumentumokat, inicializálja a komponenseket és vezérli a szimulációs ciklust.
uzzy_controller.py	A fuzzy logika alapú szabályozó implementációja. Tartalmazza a tagsági függvényeket, a szabálybázist és az inferencia mechanizmust.
pid_controller.py	A PID szabályozó implementációja a simple-pid könyvtár felhasználásával.
dc_motor_model.py	A DC motor fizikai modellje differenciálegyenletekkel. Szimulálja az elektromos és mechanikai dinamikát.
encoder_sensor.py	Az enkóder szenzor modellje, amely kvantálási hibát és mérési zajt is szimulál.
visualization.py	Matplotlib alapú vizualizációs funkciók a tagsági függvények, irányítási felület és szimulációs eredmények megjelenítésére.
motor_parameters.py	Központi konfigurációs fájl, amely tartalmazza a motor fizikai paramétereit és a szimuláció beállításait.

A szimuláció zárt szabályozási hurokként működik, amelyben minden iterációban az alábbi lépések hajtódnak végre:

1. Az enkóder méri a motor aktuális pozícióját, a mérés zajt és kvantálást is tartalmaz.
2. A szabályozó (fuzzy vagy PID) a pozícióhiba alapján kiszámítja a beavatkozó jelet.
3. A beavatkozó jel feszültséggé alakul, amely bemenetként kerül a motor modellre.
4. A motor modell frissíti a pozíciót a fizikai egyenletek alapján (dinamika, tehetetlenség, súrlódás stb.).
5. A ciklus ismétlődik, amíg a rendszer konvergál vagy el nem érjük a maximális lépésszámot.

A motor_parameters.py központi konfigurációs szerepet tölt be: ebben a modulban vannak definiálva a motor fizikai állandói, a szimuláció időlépése, az enkóder felbontása, valamint a szabályozók alapértelmezett paramétereit. Ennek köszönhetően a rendszer konfigurációja egyetlen helyen módosítható, ami egyszerűsíti a kísérletezést és a karbantartást.



16 Ábra. – Szoftver felépítése

8. Komponensek működése

A szimulációs szoftver több, jól elkülöníthető modulból épül fel. Minden komponens egy konkrét feladatot lát el: a motor modellje a fizikai dinamikát számítja, az enkóder a pozíciómérést szimulálja, a szabályozók a beavatkozó jelet állítják elő, míg a segédmodulok a logikát és az adatáramlást biztosítják. A modulok pontos működése és egymáshoz való kapcsolata határozza meg a teljes szimuláció viselkedését.

A részletes technikai dokumentáció és a forráskód a GitHub repository **README.md** fájljában található.

8.1 DC Motor Modell

A `dc_motor_model.py` modul a DC motor fizikai viselkedését szimulálja differenciálegyenletek numerikus megoldásával.

Fő jellemzők

- **Euler-módszerrel** számítja az áram, a szögsebesség és a pozíció időbeli változását.
- Figyelembe veszi a **súrlódást**, a **tehetetlenséget** és a **vissza-EMF** hatását.
- Minden `step()` híváskor frissíti a motor teljes állapotát, a bemeneti feszültség és az előző állapot alapján.

8.2. Enkóder Modell

Az `encoder_sensor.py` modul egy valós inkrementális enkódert szimulál, mérési zajjal kiegészítve.

Alapértelmezett paraméterek

- **1000 PPR** → **0.36° / impulzus** felbontás
- **0.1°** szórású Gauss-zaj a mérésben

Működés

- A folytonos pozíciót **diszkrét impulzusokra kvantálja**, az enkóder felbontásának megfelelően.
- A mérésre **Gauss-eloszlású zajt** ad a valósághű szimuláció érdekében.
- Az enkóder felbontása a **PPR (Pulses Per Revolution)** paraméter alapján számítható.

8.3 Fuzzy Szabályzó

A `fuzzy_controller.py` modul a **scikit-fuzzy** könyvtárra épül, és **Mamdani-típusú** fuzzy inferenciát alkalmaz.

Szabálybázis

A szabályozó **9 fuzzy szabályt** használ (3×3 mátrix), amelyek a két bemenet – a pozícióhiba és a hiba változása – összes kombinációjára definiálnak kimenetet. A szabályok részletes kifejtése a következő fejezetben található.

Integráló tag

A tisztán fuzzy alapú szabályozó **maradó hibát** hagyhat a beállási állapotban. Ennek kompenzálására a fuzzy kimenethez hozzáadódik egy **integráló tag**:

- integrálási erősítés: **$k_i = 0.5$**
- az integrált hiba ± 300 tartományra van **korlátozva** (anti-windup)

Ez a kiegészítés lehetővé teszi a referencia pontosabb követését, miközben megakadályozza az integrátor túlvezérlését.

8.4 PID Szabályzó

A `pid_controller.py` modul a **simple-pid** könyvtárat használja, amely egy megbízható, ipari jellegű PID-implementációt biztosít.

Kimeneti korlátozás

A PID-szabályozó kimenete **-100 és +100** tartományra van korlátozva. Ez megegyezik a fuzzy szabályzó kimeneti tartományával, így a két szabályozási módszer **közvetlenül összehasonlítható** ugyanabban a szimulációs környezetben.

8.5 Vizualizáció

A **visualization.py** modul matplotlib alapú grafikus megjelenítést biztosít, amely lehetővé teszi a motorválasz, a szabályozó jelek és a szimuláció különböző állapotváltozóinak vizuális elemzését.

Elérhető ábrák:

<code>plot_membership_functions()</code>	Tagsági függvények	Fuzzy
<code>plot_control_surface()</code>	3D vezérlési felület	Fuzzy
<code>plot_simulation_results()</code>	4 részes eredmény ábra	Fuzzy
<code>plot_pid_results()</code>	4 részes eredmény ábra	PID
<code>plot_final_summary()</code>	Összesítő oszlopdiagram	Mindkettő

A 4 részes ábra tartalma:

- Pozíció vs idő
- Hiba vs idő
- Vezérlőjel vs idő
- Vezérlőjel vs hiba

8.6 Motor Paraméterek

A **motor_parameters.py** modul **központosítja a szimuláció összes beállítását**, beleértve a motor fizikai paramétereit, a szabályozók alapértékeit, az enkóder felbontását és az időlépést. Ennek köszönhetően a rendszer **könnyen hangolható**, mivel minden konfigurációs érték egyetlen helyen módosítható.

Motor fizika	J, K _f , K _m , R, L
Szimuláció	DT, MAX_STEPS, küszöbök
Fuzzy tartományok	hiba, delta_hiba, vezérlőjel határok
Enkóder	PPR, zajszórás

9. Fuzzy Szimuláció működése

A korábban ismertetett fuzzy elméleti alapok ebben a fejezetben öltönek testet a szimulációs környezetben. Először meghatározzuk a bemeneti és kimeneti univerzumokat, majd definiáljuk a hozzájuk tartozó tagsági függvényeket és összeállítjuk a szabálybázist. Végül bemutatjuk, hogyan kapcsolódnak össze ezek az elemek a valós idejű szabályozási hurokkal, és hogyan járulnak hozzá a rendszer teljes működéséhez.

9.1 Univerzumok definiálása

A fuzzy szabályzó három nyelvi változóval dolgozik, amelyek mindegyikéhez egy-egy univerzum (értelmezési tartomány) tartozik. Az univerzumok határozzák meg, hogy az adott változó milyen értéktartományban mozoghat, ezáltal keretet adnak a tagsági függvények és a későbbi fuzzy szabályok definiálásához.

Bemeneti univerzumok

Hiba (error) univerzum:

A pozícióhiba a célpozíció és az aktuális pozíció különbsége, fokban kifejezve. Az univerzum tartománya -180° és $+180^\circ$ között mozog, így lefedi a teljes lehetséges pozícióeltérést, amely a szabályozó bemenetét képezi.

Paraméter	Érték	Mértékegység
Minimum	-180	fok
Maximum	180	fok
Felbontás	1	fok

Hibaváltozás (delta_error) univerzum:

A hibaváltozás azt mutatja meg, hogy a hiba milyen gyorsan és milyen irányba módosul az egymást követő mintavételek között. Ez a bemenet segíti a szabályzót a rendszer közeljövőbeli viselkedésének „előrejelzésében”, így fontos szerepet játszik a stabilitás és a túllendülés csökkentésében. Az univerzum tartománya a szimuláció igényeihez igazodva keretezi a lehetséges értékeket.

Paraméter	Érték	Mértékegység
Minimum	-50	fok
Maximum	+50	fok
Felbontás	1	fok

Kimeneti univerzumok

Vezérlőjel (control) univerzum:

A szabályzó kimenete egy dimenziómentes vezérlőjel, amely a szabályozási döntést numerikus formában fejezi ki. Ezt az értéket a szimuláció későbbi lépéseiben feszültséggé alakítjuk, amely közvetlenül a motor meghajtására szolgál. Az univerzum tartománya határozza meg, hogy a szabályzó milyen erősségű beavatkozást alkalmazhat.

Paraméter	Érték	Mértékegység
Minimum	-100	-
Maximum	+100	-
Felbontás	1	-

9.2 Tagsági Függvények

Minden univerzumon három fuzzy halmaz kerül definiálásra, amelyek a nyelvi értékeket reprezentálják: **Negatív (N)**, **Zéró (Z)** és **Pozitív (P)**. A tagsági függvények alakja háromszög vagy trapéz, amelyek átfedése biztosítja a fokozatos átmenetet a halmazok között.

A szélső értékeket leíró trapéz alakú halmazok a tartomány szélein telítésbe mennek, azaz elérik a maximális tagsági értéket. A háromszög alakú Zéró halmaz pedig a nulla körüli kis hibák finom kezelését teszi lehetővé, így biztosítva a precíz szabályozást a beállási tartományban.

Hiba Tagsági Függvényei

Halmaz	Típus	Paraméterek
N (Negatív)	Trapéz	[-180, -180, -30, -5]
Z (Zero)	Háromszög	[-8, 0, 8]
P (Pozitív)	Trapéz	[5, 30, 180, 180]

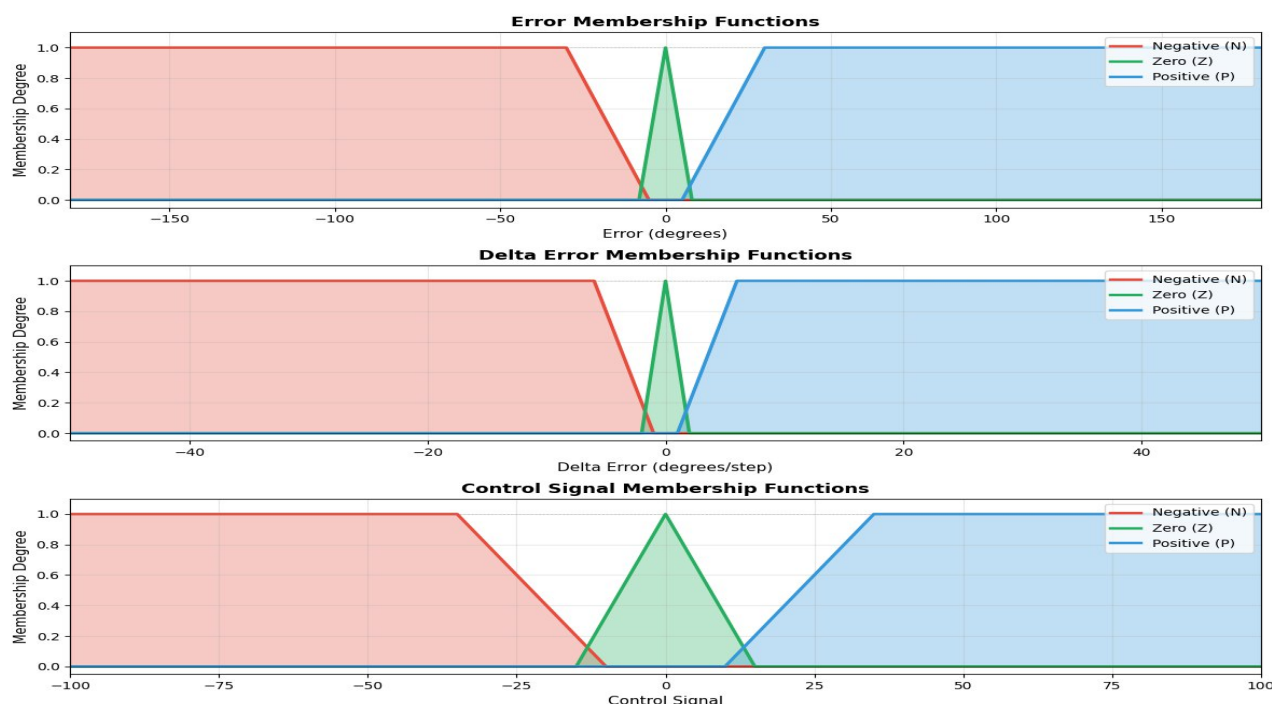
Hibaváltozás és vezérlő jel tagsági függvényei

Hibaváltozás:

Halmaz	Típus	Paraméterek
N (Negatív)	Trapéz	[-50, -50, -6, -1]
Z (Zero)	Háromszög	[-2, 0, 2]
P (Pozitív)	Trapéz	[1, 6, 50, 50]

Vezérlőjel:

Halmaz	Típus	Paraméterek
N (Negatív)	Trapéz	[-100, -100, -35, -10]
Z (Zero)	Háromszög	[-15, 0, 15]
P (Pozitív)	Trapéz	[10, 35, 100, 100]



17 Ábra. – Fuzzy Tagsági Függvények

9.3 Szabálybázis

A szabályzó egy 3×3-as szabálybázist alkalmaz, amely minden lehetséges bemeneti kombinációhoz meghatározza a megfelelő kimeneti fuzzy halmazt. A szabályok IF-THEN formában vannak megfogalmazva, ahol a két bemenet – a hiba és a hibaváltozás – között ÉS (AND) kapcsolat áll fenn, így egy szabály akkor aktiválódik, ha mindkét feltétel bizonyos mértékben teljesül.

	delta_error/error = N	delta_error/error = Z	delta_error/error = P
error = N	control = N	control = N	control = Z
error = Z	control = Z	control = Z	control = Z
error = P	control = Z	control = P	control = P

- **Ha a hiba negatív** (a rendszer túllőtt a célon), és a hibaváltozás nem csökken vagy tovább romlik, akkor **negatív beavatkozás szükséges** a visszatéréshez.
- **Ha a hiba kicsi** (a Zéró tartományban van), akkor a beavatkozásnak is **kicsinek kell maradnia** a stabilitás fenntartása érdekében.
- **Ha a hiba pozitív** (a cél még nem érhető el), és a hibaváltozás nem csökken, akkor **pozitív beavatkozás szükséges** a cél felé gyorsításhoz.
- **Ha a hiba és a hibaváltozás ellentétes előjelű**, akkor a rendszer már a helyes irányba kezdett elmozdulni, ezért **mérsékeltebb beavatkozás is elegendő**, hogy elkerüljük a túllendülést.

9.4 Integráló tag és végső kimenet

A tisztán fuzzy szabályzó mellett egy integrális (I) tag is hozzáadódik a kimenethez. Ez a kiegészítés segít kiküszöbölni a maradó szabályozási hibát (steady-state error), amely a fuzzy szabályzóknál gyakran megjelenhet. A végső kimenet képlete:

$$u(tk)_{\text{Fuzzy}} = \text{Fuzzy_Kimenet}(tk) + K_i(tk) * I(tk)$$

Az integrál értéke -300 és $+300$ között van korlátozva, hogy elkerüljük az integrátor felfutását (windup) hosszabb tranzien্স folyamatok során. Az integrális erősítés értéke:

$$K_i = 0.5$$

Ez a kombinált fuzzy-I struktúra pontosabb beállási értéket eredményez, miközben megőrzi a fuzzy szabályozás rugalmasságát és robusztusságát.

9.5 Szimulációs ciklus

A szimuláció a `simulate_motor_control_fuzzy()` függvényben valósul meg. A ciklus minden iterációja a valós idejű szabályozási hurok egy lépését modellezi. Az alábbiak történnek:

Inicializálás

A szimuláció indulásakor létrejönnek a szükséges komponensek:

- DC motor modell a kezdeti pozícióval
- Enkóder szenzor a valósághű pozícióméréshez
- Fuzzy szabályzó a beavatkozó jel kiszámításához

Szabályozási hurok lépései:

1. **Hibaképzés**
A célpozíció és a mért pozíció különbségének kiszámítása.
2. **Hibaváltozás számítása**
Az aktuális hiba és az előző iteráció hibájának különbsége.
3. **Fuzzy inferencia**
A szabályzó kiszámítja a beavatkozó jelet a fuzzifikáció → szabálykiértékelés → defuzzifikáció lépésein keresztül.
4. **Feszültségkonverzió**
A kimeneti vezérlőjel átszorzása a motor meghajtásához szükséges feszültségszkalázó tényezővel.
5. **Motor léptetése**
A motor fizikai modelljének frissítése a kapott bemenő feszültség alapján (áram, szögsebesség, pozíció).
6. **Pozíció mérés**
Az enkóder leolvassa az új pozíciót, zajjal és kvantálással terhelve.

Az aktuális állapot (pozíció, hiba, vezérlőjel stb.) mentésre kerül a későbbi vizualizációhoz.

Konvergencia vizsgálat

A szimuláció akkor áll le, ha a rendszer konvergált a célpozícióhoz. A leállás feltételei:

- A pozícióhiba abszolút értéke kisebb mint 0.5°
- A hibaváltozás abszolút értéke kisebb mint 0.5°

Ha mindkét feltétel teljesül, a szabályzó elérte a célját, és a szimuláció befejeződik. Ha bármelyik nem teljesül, a ciklus tovább folytatódik a maximális lépésszám eléréséig.

9.6 Eredmények vizualizációja

1. Tagsági függvények ábrázolása

- A három univerzum (hiba, hibaváltozás, vezérlőjel) tagsági függvényeinek grafikus megjelenítése.

2. Szabályzási felület (control surface)

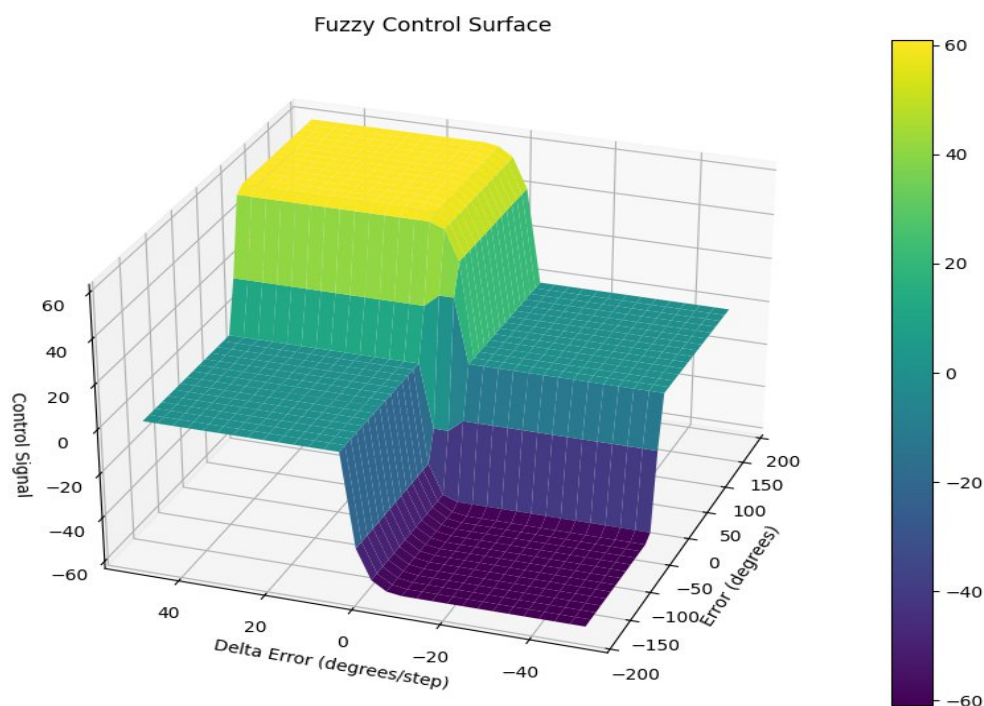
- Egy 3D felület, amely megmutatja, hogy a két bemenet különböző kombinációira milyen kimenetet ad a fuzzy rendszer.

3. Időbeli lefutás

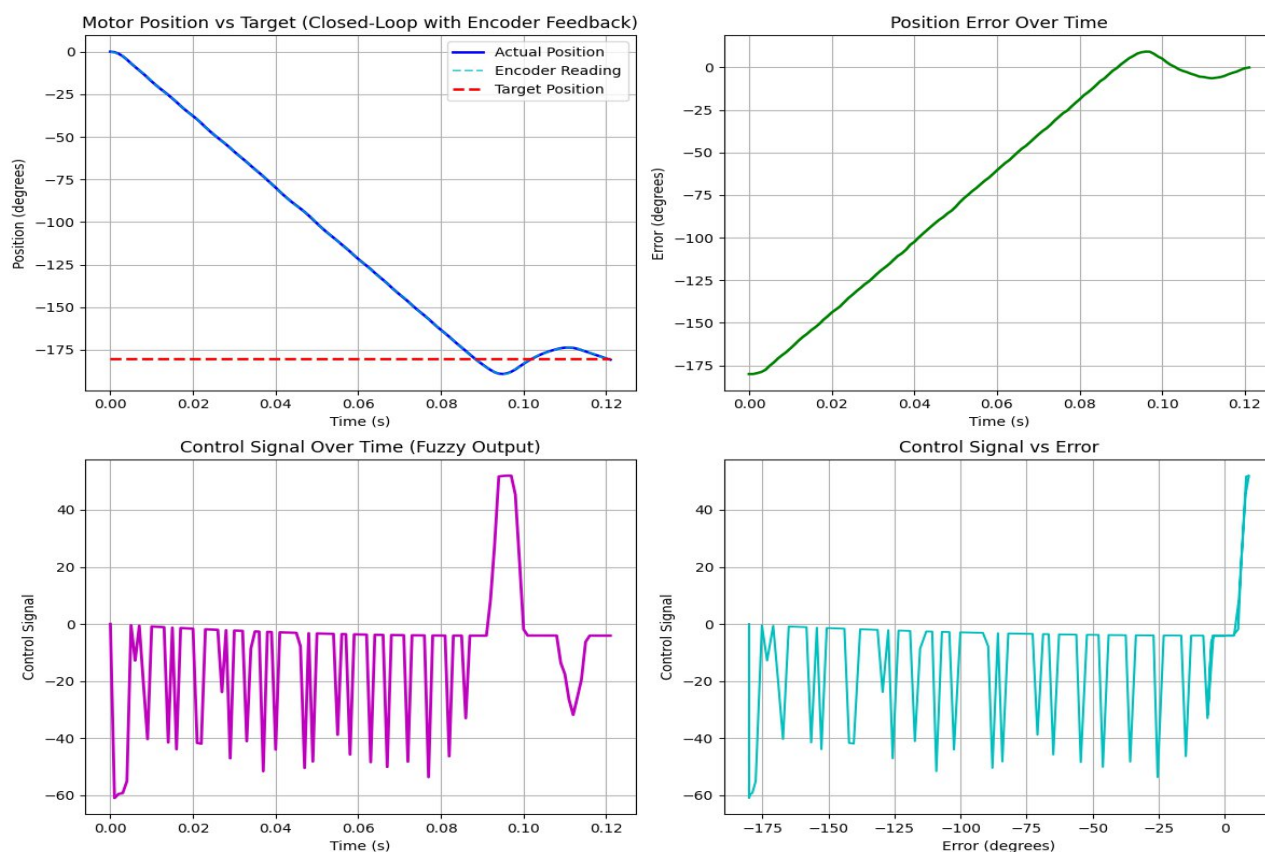
- A pozíció, a hiba és a vezérlőjel alakulása az idő függvényében, amely megmutatja a beállási folyamatot és a rendszer dinamikáját.

4. Összefoglaló

- A szimuláció végeredményének összesítése (pl. beállási idő, túllövés mértéke, konvergencia).

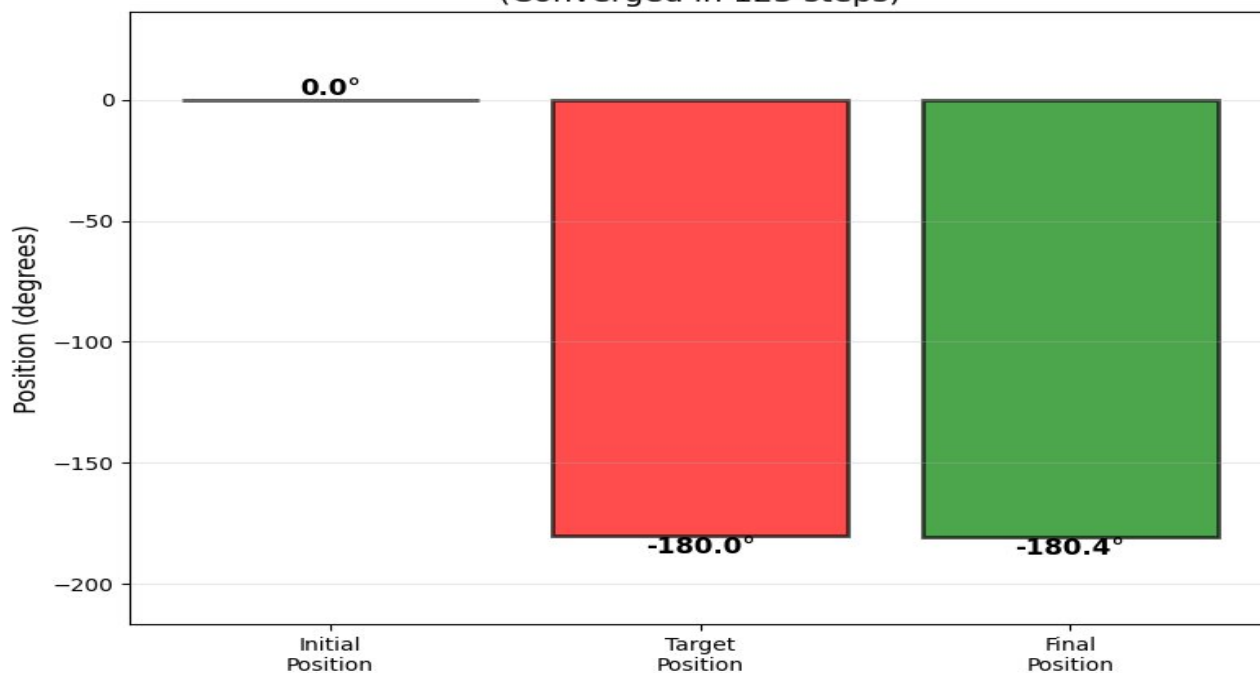


18 Ábra. – Fuzzy Szabályzó felület



19 Ábra. – Fuzzy Szabályzó kimenete

Motor Position Control Summary (Fuzzy) (Converged in 123 steps)



20 Ábra. – Fuzzy Szabályzás Végeredménye

10. PID Szimuláció működése

A PID szabályzás a fuzzy megközelítéssel ellentétben matematikai képleteken alapul, és a rendszer dinamikus viselkedését közvetlenül a P, I és D tagok számított értékei határozzák meg. A szimulációs környezet lehetővé teszi a két módszer közvetlen összehasonlítását, mivel mindkét szabályozó azonos kezdeti feltételek és paraméterek mellett vizsgálható.

10.1 Szabályozó paraméterei

A szimulációban használt PID szabályzó három hangolási paraméterrel rendelkezik:

Paraméter	Jelölés	Alapértelmezett érték	A pillanatnyi hibára adott válasz erőssége
Proporcionális tag	Kp	2.0	A pillanatnyi hibára adott válasz erőssége
Integrálás tag	Ki	0.5	A felhalmozódott hiba kompenzálása
Derivált tag	Kd	0.1	A hibaváltozás csillapítása

A vezérlőjel kimenet korlátozva van -100 és +100 közötti tartományra, megegyezően a fuzzy szabályzó kimeneti univerzumával. Ez biztosítja az összehasonlíthatóságot a két módszer között.

10.2 Szimulációs ciklus

A `simulate_motor_control_pid()` függvény struktúrája megegyezik a fuzzy változattal. A különbség a szabályzó működésében rejlik.

Inicializálás

A PID szabályzó létrehozásakor beállításra kerülnek:

- A három erősítési paraméter: Kp, Ki, Kd
- A kimeneti korlátok: -100 ... +100
- A célpozíció (setpoint)

Szabályozási hurok

1. Hibaképzés

- A célpozíció és a mért pozíció különbségének kiszámítása.

2. PID számítás

- A simple-pid könyvtár kiszámítja a három PID-tag (P, I, D) összegét, amely a beavatkozó jelet adja.

3. Feszültségkonverzió

- A vezérlőjel átszorítása a motor meghajtásához szükséges feszültség-skálázási tényezővel.

4. Motor léptetése

- A fizikai motormodell frissítése az alkalmazott bemenő feszültség alapján.

5. Pozíció mérése

- Az enkóder leolvassa az aktuális pozíciót, zajjal és kvantálással terhelve.

Konvergencia feltételek

- A leállási feltételek azonosak a fuzzy szimulációval:
- Pozícióhiba kisebb mint 0.5°
- Hibaváltozás kisebb mint 0.5°

10.3 Különbségek a fuzzy szimulációhoz képest

Szempont	Fuzzy szabályzó	PID szabályzó
Bemenetek	hiba, hibaváltozás	csak mért pozíció
Számítás módja	szabálybázis + inferencia	matematikai modell
Hangolás	tagsági függvények, szabályok	három numerikus paraméter
Vizualizáció	tagsági függvények, szabályzási felület	csak időbeli lefutás

10.4 Eredmények vizualizációja

A PID szimuláció végén a `plot_pid_results()` függvény jeleníti meg az eredményeket. A fuzzy verzióval ellentétben itt nem láthatók tagsági függvények vagy vezérlési felület; kizárólag az időbeli lefutási görbék kerülnek ábrázolásra.

A vizualizáció tartalma:

1. Pozíció alakulása

- A motor pozíciójának időbeli változása a célértékhez viszonyítva.

2. Szabályozási hiba változása

- A hiba időfüggvénye, amely megmutatja, hogyan közelíti a rendszer a referenciaértéket.

3. Szabályozási jel időbeli lefutása

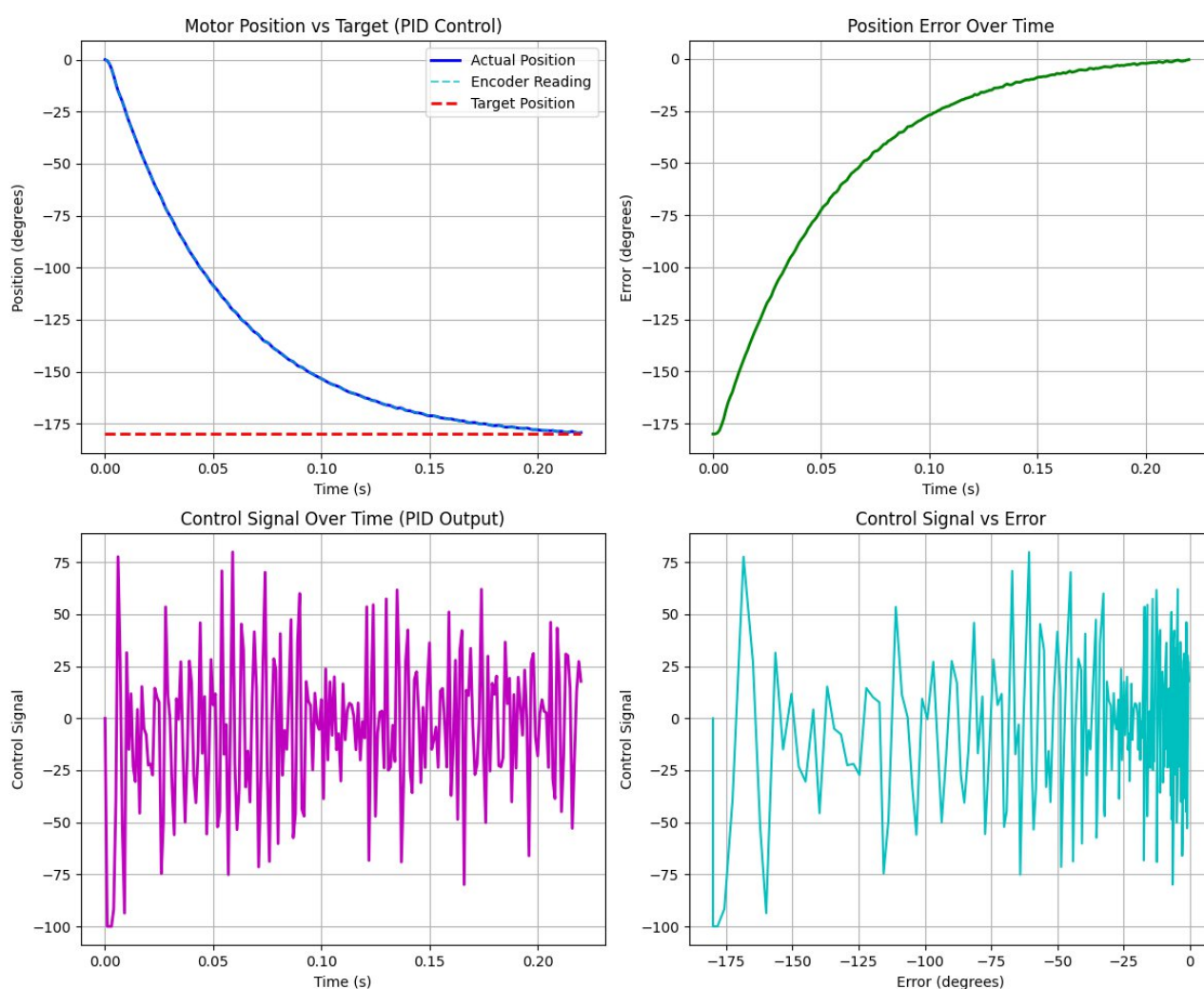
- A PID kimenetének változása az idő függvényében (beavatkozó jel).

4. Mért és valós pozíció összehasonlítása

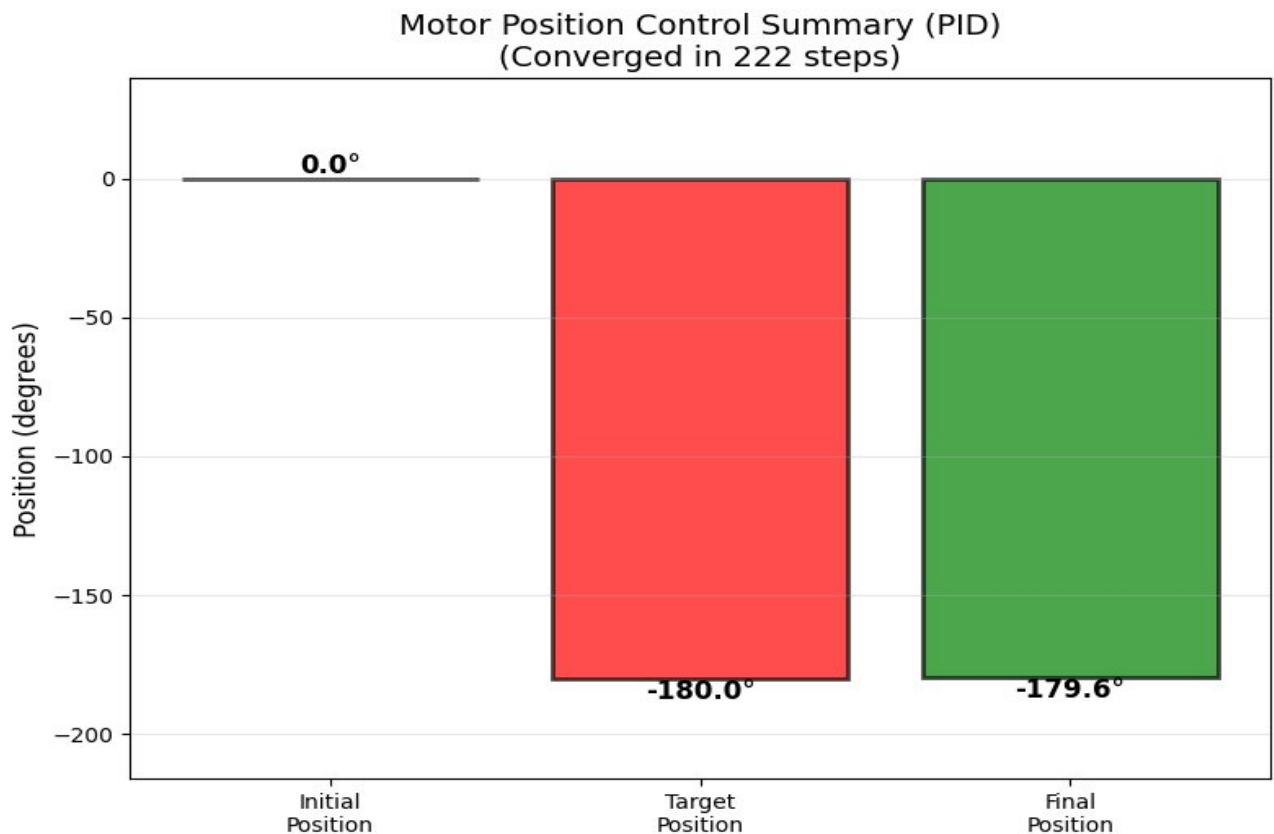
- A zajjal terhelt enkóderpozíció és a valós modellpozíció együttes megjelenítése.


```
def __init__(self, kp=2.0, ki=0.5, kd=0.1):  
    """  
    Initialize PID controller with tuning parameters.  
  
    Args:  
        kp: Proportional gain  
        ki: Integral gain  
        kd: Derivative gain  
    """
```

21 Ábra. – PID Paraméterek



22 Ábra. – PID Szabályzás Kimenete



23 Ábra. – PID Szabályzás Végeredménye

```
Starting closed-loop simulation with encoder feedback:
Controller: PID
Encoder resolution: 1000 PPR (0.360°/count)
Encoder noise: 0.1° std dev
Initial position: 0.0°
Target position: -180.0°
Initial error: -179.9746738622087°
Time step: 1.00 ms

Step 20 (0.020s): Actual=-53.03°, Measured=-52.98°, Err=-130.04°, V=-4.56V, Count=-147
Step 40 (0.040s): Actual=-93.32°, Measured=-93.21°, Err=-88.46°, V=-5.20V, Count=-259
Step 60 (0.060s): Actual=-120.90°, Measured=-120.94°, Err=-59.67°, V=0.35V, Count=-336
Step 80 (0.080s): Actual=-140.47°, Measured=-140.54°, Err=-40.72°, V=0.24V, Count=-390
Step 100 (0.100s): Actual=-153.76°, Measured=-153.59°, Err=-27.08°, V=1.77V, Count=-427
Step 120 (0.120s): Actual=-162.62°, Measured=-162.67°, Err=-18.04°, V=-0.40V, Count=-452
Step 140 (0.140s): Actual=-168.61°, Measured=-168.58°, Err=-12.06°, V=-4.32V, Count=-468
Step 160 (0.160s): Actual=-172.85°, Measured=-172.88°, Err=-7.08°, V=-1.36V, Count=-480
Step 180 (0.180s): Actual=-176.07°, Measured=-175.97°, Err=-4.34°, V=1.31V, Count=-489
Step 200 (0.200s): Actual=-177.81°, Measured=-177.75°, Err=-2.14°, V=-1.29V, Count=-494
Step 220 (0.220s): Actual=-179.54°, Measured=-179.43°, Err=-0.94°, V=-2.31V, Count=-499

Converged at step 222 (0.222s)!
Final actual position: -179.61°
Final measured position: -179.61°
Final error: -0.48°
Final velocity: 87.47°/s
Final current: 0.0731 A
Final encoder count: -499
```

24 Ábra. – PID Szabályzás Működése

11. Összegzés és jövőbeli irányok

11.1 Rendszer értékelése

A megvalósított szimuláció sikeresen demonstrálja a zárt hurkú szabályozás működését. A rendszer főbb jellemzői:

Szempont	Megvalósítás
Motor modell	Fizikailag realiztikus, differenciálegyenlet-alapú
Érzékelés	Enkóder modell zajjal
Fuzzy szabályzó	3x3 szabálybázis, integrális kiegészítéssel
PID szabályzó	Klasszikus P-I-D tagok, anti-windup
Vizualizáció	Részletes grafikonok, tagsági függvények

Mindkét szabályzó képes a motor pozícióját a célértékre állítani a megadott konvergenciakritériumokon belül. A fuzzy szabályzó előnye, hogy az emberi gondolkodáshoz közel álló szabályokkal működik, míg a PID szabályzó matematikailag egyszerűbb és könnyebben hangolható.

A szimuláció oktatási és demonstrációs célokra készült; a hangsúly a megérthetőségen és az átláthatóságon volt. A kód moduláris felépítése lehetővé teszi az egyes komponensek önálló tanulmányozását, módosítását és továbbfejlesztését.

11.2 Jövőbeli fejlesztési irányok

A szimulációs rendszer számos irányban bővíthető és fejleszthető. Az alábbi területek különösen ígéretesek a továbbfejlesztés szempontjából.

Adaptív szabályzók

A jelenlegi megoldásban a szabályzók paraméterei – PID esetén **K_p**, **K_i**, **K_d**, fuzzy esetén a **tagsági függvények** és a **szabálybázis** – manuálisan kerülnek beállításra. Egy adaptív szabályozási rendszer képes lenne ezeket a paramétereket **automatikusan hangolni** a rendszer válaszána elemzése alapján.

Lehetséges megközelítések:

- **Ziegler-Nichols** automatikus hangolás implementálása
- **Model Reference Adaptive Control (MRAC)**
- **Genetikus algoritmus** alapú paraméteroptimalizálás

Hibrid Fuzzy-PID szabályzó

A két szabályzó típus kombinálásával kihasználhatók mindkét megközelítés előnyei. A hibrid rendszerek célja, hogy a fuzzy logika rugalmasságát és emberi jellegű döntéshozását összevonják a PID szabályzó matematikai stabilitásával és egyszerű hangolhatóságával.

Lehetséges működési elvek:

- A fuzzy rendszer dinamikusan állítja a PID paramétereket a hiba nagyságától vagy irányától függően.
- Nagy hiba esetén agresszívebb, kis hiba esetén finomabb szabályzás valósulhat meg.

- Nemlineáris rendszerek esetén a hibrid megközelítés gyakran jobb teljesítményt és stabilitást ad, mint a tisztán PID vagy tisztán fuzzy megoldás.

Zavar és terhelésváltozás kezelése

A jelenlegi modell nem tartalmaz külső zavarokat vagy terhelésváltozásokat. A rendszer továbbfejleszthető úgy, hogy a valós környezetet jobban megközelítse.

Lehetséges bővítési irányok:

- Változó terhelés szimulációja (időfüggő vagy nemlineáris terhelő nyomaték alkalmazása)
- Külső zavarok injektálása, például impulzusszerű vagy folyamatos terhelési zavarok
- Zavar-elutasító szabályzó tervezése, például feedforward kompenzáció vagy robusztus szabályozási módszerek alkalmazása

Neurális hálózat alapú szabályzás

A modern gépi tanulási módszerek integrálása tovább növelheti a szabályozó rendszer alkalmazkodóképességét és teljesítményét, különösen nemlineáris vagy erősen változó környezetekben.

Lehetséges irányok:

- Megerősítéses tanulás (Reinforcement Learning) alapú szabályzó, amely jutalmazási rendszer alapján sajátítja el az optimális beavatkozási stratégiát
- Neurális hálózattal approximált fuzzy tagsági függvények, amelyek automatikusan finomítják a fuzzy rendszer bemeneti-kimeneti leképezését
- ANFIS (Adaptive Neuro-Fuzzy Inference System), amely ötvözi a fuzzy logika értelmezhetőségét a neurális hálók tanulási képességével

Valós hardver és firmware/szoftver implementáció

A szimulációból valós hardveres rendszerbe történő átmenet lehetőséget ad a szabályozási algoritmusok gyakorlati kipróbálására és finomhangolására.

Lehetséges megvalósítási irányok:

- Arduino vagy Raspberry Pi alapú vezérlőrendszer létrehozása
- Valós DC motor és enkóder csatlakoztatása és visszacsatolási kör kiépítése
- PWM jelgenerálás és H-híd meghajtás implementálása a motor teljesítményvezérléséhez

Grafikus felhasználói felület

A jelenlegi parancssori interfész helyett egy grafikus felület fejlesztése jelentősen javíthatná a rendszer használhatóságát és szemléltethetőségét.

Lehetséges funkciók:

- Paraméterek valós idejű állítása (motorparaméterek, PID/Fuzzy beállítások)
- Élő grafikonok a szabályozási folyamat megjelenítésére
- Különböző motorparaméter-készletek betöltése és összehasonlítása
- Szabályzók párhuzamos összehasonlítása egyidejű grafikus megjelenítéssel

11.3 Szimuláció korlátai

A jelenlegi implementáció több egyszerűsítéssel él, amelyek figyelembevétele fontos a modell értelmezésekor.

Fő korlátok:

- A motor modell **ideális körülményeket** feltételez (állandó hőmérséklet, nincs mechanikai kopás vagy csapágyvesztés).
- Az enkóder modell **egyszerűsített zajkarakterisztikával** dolgozik, amely nem minden valós szenzorviselkedést tükröz.
- A szimulációs **lépésköz 1 ms**, amely valós alkalmazásokban gyakran finomabb időfelbontást igényelhet.
- A szabályzók nem kezelnek **szaturációt**, holtzónát vagy egyéb **nemlinearitásokat**, amelyek a fizikai rendszerben jelen vannak.

Ezek a korlátok oktatási célra megfelelőek, azonban **valós rendszer tervezésekor** mindenképpen figyelembe kell venni őket.

11.4 A Projekt Összegzése

A bemutatott szimulációs rendszer egy átfogó és jól strukturált keretet biztosít a DC motorok pozíciószabályozásának tanulmányozásához. A modellben különválasztottuk a motor fizikai viselkedését, a mérőrendszer karakterisztikáját és a szabályzó működését, így a felépítés nemcsak átlátható, hanem könnyen bővíthető is. A fuzzy logikán alapuló és a PID szabályozási módszerek párhuzamos vizsgálata lehetővé tette, hogy a két megközelítés előnyeit és korlátait közvetlenül összehasonlítsuk.

A PID szabályzó gyors reakciót és matematikailag jól értelmezhető, stabil viselkedést biztosít, miközben hangolása viszonylag egyszerű marad. A fuzzy szabályzó ezzel szemben rugalmasabb, a nemlineáris jelenségekkel szemben toleránsabb működést tesz lehetővé, és képes emberközelű szabályok alapján döntést hozni. A két megközelítés egymás mellé helyezése rávilágít arra, hogy valós rendszerekben az optimális teljesítmény gyakran hibrid megoldásokkal érhető el.

A szimuláció moduláris felépítése lehetővé teszi további fejlesztések integrálását, például adaptív szabályzókat, zavar- és terhelésmoделl beépítését, vagy akár neurális hálózatok alkalmazását. Ezek az irányok fontos szerepet játszanak a modern irányítástechnikában, ahol a rendszerek egyre komplexebbek és egyre több bizonytalansággal terheltek. A jelenlegi implementáció oktatási célokra kiválóan alkalmas, mivel egyszerre szemlélteti a szabályzástechnika alapfogalmait és ad betekintést a haladó módszerek irányába is.

Összességében a projekt értékes kiindulópontot jelent mind a gyakorlati megvalósítás, mind a további kutatás és fejlesztés számára. A modell bővíthető, a kód jól áttekinthető, a vizualizációk pedig segítenek a dinamikus viselkedés intuitív megértésében. A feldolgozott témák széles köre lehetővé teszi, hogy a rendszer később valós hardveren is kipróbálható legyen, ami tovább erősíti az oktatási és kutatási értékét.

12. Idegen nyelvű tartalmi összefoglaló

The presented simulation framework provides a comprehensive and well-structured environment for studying position control of DC motors. The model clearly separates the physical behavior of the motor, the characteristics of the measurement system, and the operation of the controllers, resulting in a modular architecture that is both transparent and easily extendable. By examining the PID and fuzzy logic controllers in parallel, the system enables a direct comparison of their respective advantages and limitations.

The PID controller offers fast response, mathematically well-defined behavior, and straightforward tuning while maintaining stability. In contrast, the fuzzy controller provides greater flexibility, improved tolerance to nonlinearities, and the ability to make decisions based on human-interpretable rules. Comparing the two methods highlights that real-world control systems often achieve optimal performance through hybrid approaches that combine the strengths of both techniques.

The modular structure of the simulation allows further extensions, such as adaptive control strategies, disturbance and load modeling, or even machine learning-based methods like neural networks. These directions play an increasingly important role in modern control engineering, where systems are becoming more complex and subject to higher uncertainty. The current implementation is well suited for educational purposes, as it demonstrates fundamental principles of control theory while also offering a stepping stone toward more advanced methods.

Overall, the project serves as a valuable starting point for practical implementation as well as further research and development. The model is extendable, the source code is clear and maintainable, and the visualizations support intuitive understanding of the system's dynamic behavior. The broad range of covered topics makes the framework suitable for future testing on real hardware, further enhancing its educational and research relevance.

13. Felhasznált irodalom

- [1] K. J. Åström, R. M. Murray: *Feedback Systems – An Introduction for Scientists and Engineers*. Princeton University Press, 2010.
- [2] O. Nelles: *Nonlinear System Identification – From Classical Approaches to Neural Networks and Fuzzy Models*. Springer, 2001.
- [3] Zadeh, L. A.: “Fuzzy Sets.” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [4] Ross, T. J.: *Fuzzy Logic with Engineering Applications*. Wiley, 2010.
- [5] D. E. Seborg, T. F. Edgar, D. A. Mellichamp: *Process Dynamics and Control*. Wiley, 2016.
- [6] B. C. Kuo: *Automatic Control Systems*. Prentice Hall, 1995.
- [7] simple-pid Python Library – Documentation.
<https://github.com/m-lundberg/simple-pid>
- [8] scikit-fuzzy Library – Documentation.
<https://pythonhosted.org/scikit-fuzzy/>
- [9] Python Software Foundation. *Python 3 Documentation*.
<https://docs.python.org/3/>
- [10] Wikipedia contributors: “DC motor.” *Wikipedia, The Free Encyclopedia*.
https://en.wikipedia.org/wiki/DC_motor
- [11] Wikipedia contributors: “Fuzzy logic.” *Wikipedia, The Free Encyclopedia*.
https://en.wikipedia.org/wiki/Fuzzy_logic
- [12] A projekt GitHub repozitóriuma (saját készítés):
<https://github.com/Ricsi1231/DC-Motor-Position-Control-Fuzzy-Simulation>